

DRISHTISCAN

Prototype Development Lifecycle & Technical Report



An engineering record of the prototype, its processing pipeline, safety controls, performance work, verification strategy, and present limitations.

Smart India Hackathon 2026 - SIH26034

1. Executive Summary

DrishtiScan is a software-assisted compliance scanner for packaged commodities. It turns information printed on product packaging into structured declarations, keeps the evidence used to obtain those declarations, checks the resulting fields with deterministic Legal Metrology logic, and produces a human-readable compliance result.

Core principle

Preserve evidence → reconstruct structure → classify and map → validate against evidence → evaluate rules → report uncertainty when evidence is insufficient.

OCR and semantic structuring operate on observed packaging evidence. The compliance layer is separate: it evaluates normalized fields against encoded rules. This keeps the final decision path auditable rather than making an AI model the source of legal policy.

The current implementation is organized as a web frontend, Node/Express orchestration layer, independent OCR service, spatial reconstruction and multi-photo fusion, GPT-OSS structuring, deterministic grounding, rule evaluation, and report generation. Reliability features sit across the same path: upload validation, rate limiting, explicit OCR failure handling, conflict detection, deterministic fallback, and scan tracing.

2. Problem Context

Packaged-product labels combine many declarations in small, crowded, and differently formatted areas. A single package can contain identity text, quantities, prices, dates, batch and licence numbers, addresses, origin statements, ingredients, nutrition information, instructions, and marketing text.

- Manual inspection combines visual reading, transcription, interpretation, and rule-by-rule checking.
- The same nearby number can mean a price, quantity, dosage, serving value, date, or code depending on its spatial context.
- An incorrect value is especially risky when it is displayed as a definite PASS instead of being surfaced for review.
- The system therefore treats false confidence as a key safety concern, not just raw OCR accuracy.

3. Users and Intended Use

User	Intended use
Compliance / field officer	First-pass inspection, structured findings, evidence-oriented report
Consumer	Quick view of visible declarations and compliance indicators
Manufacturer / packer	Pre-release packaging verification
Marketplace / e-commerce	Possible future screening or API-based pre-checking

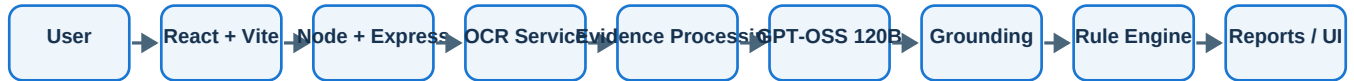
4. End-to-End Workflow



- Images are uploaded as multipart data.
- The backend sends photo OCR jobs concurrently.
- PaddleOCR returns text, confidence, and bounding geometry.
- Rows and cells are reconstructed from the 2D layout.
- Repeated evidence across photos is fused into canonical rows.

- One GPT-OSS 120B structuring call is used when available.
- Returned fields are checked against canonical OCR evidence.
- The rule engine produces PASS, POTENTIAL_NON_COMPLIANCE / FAIL, REVIEW, or INSUFFICIENT_EVIDENCE findings.
- Results can be rendered to PDF/DOCX and retained in scan traces.

5. System Architecture



The backend is the orchestration point. OCR is kept independent because it is the heaviest runtime workload and has different resource characteristics from ordinary API traffic. Evidence processing stays between OCR and semantic interpretation so that the model receives structured, geometry-aware evidence instead of a flat text dump.

Architecture responsibilities

Layer	Responsibility
Frontend	Upload, scan interaction, result display, report access
Backend	API orchestration and application workflow
OCR service	Text recognition, confidence, and geometry
Reconstruction	Visual rows, cells, reading order, normalized geometry
Fusion	Cross-photo consolidation and conflict preservation
Semantic structuring	Interpret fragmented evidence into named fields
Grounding	Verify that proposed values are supported by OCR evidence
Rule engine	Deterministic compliance evaluation
Data / storage	Rules, inspections, and evidence images
Observability	Scan traces and stage-level diagnostics

6. Input and API Layer

The backend supports separate consumer and officer scan workflows. Both accept multiple package images and share the OCR, extraction, fusion, and rule-evaluation pipeline.

Consumer scan - POST [/api/consumer/scan](#)

- Multi-image upload, validation, rate limiting, concurrent OCR, evidence extraction and fusion.
- Compliance rule evaluation and preference evaluation.
- Structured response plus optional debug trace.

Officer scan - POST [/api/officer/scan](#)

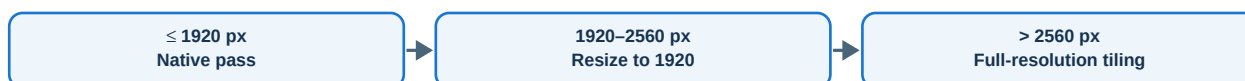
- Authenticated workflow with the same core scan path.
- Explicit multi-photo conflict findings.
- Optional inspection persistence and report generation.
- Debug trace support for forensic review.

OCR failures are returned as an explicit unavailable-service response rather than silently converting the scan into an empty report. This distinction prevents an infrastructure problem from appearing as a label with every field missing.

7. OCR Architecture

The OCR service uses FastAPI with PaddleOCR 3.7+ and the PP-OCRv6 Medium model on CPU. Text-line orientation is enabled, CLAHE is used before inference, and the recognition threshold is kept low enough to retain borderline text for downstream validation.

Resolution-aware routing



- Standard images are processed in a single native-resolution pass.
- Moderately large phone images are reduced to 1920 px to limit unnecessary CPU work.
- Ultra-large images retain full resolution and are divided into overlapping tiles so small statutory text is not lost.
- Bounding boxes are mapped back to the original image coordinates when scaling is used.
- Overlapping tile detections are deduplicated using text comparison and bounding-box overlap.

8. Spatial Reconstruction



Raw OCR output is not treated as a single string. Each detection keeps its text, confidence, index, and geometry. The reconstruction layer groups visually aligned detections into rows, orders elements left-to-right, preserves column information, and generates row text without discarding the underlying cell structure.

- Bounding-box height and overlap are used to accommodate different font sizes on the same visual line.
- Two-column layouts are kept structurally separate where the geometry indicates a meaningful gap.
- Split quantity/unit and other fragmented declarations can remain available as separate cells.
- The result is a row object containing text, elements/cells, geometry, and reading-order information.

9. Multi-Photo Evidence Fusion



Multiple photographs are treated as separate observations of the same package. The fusion layer does not infer fields. It decides whether observations can safely share a canonical evidence row and keeps provenance for every contributing photo.

Merge behavior

Condition	Handling
Exact normalized match	Safe to merge

Condition	Handling
Materially different numeric tokens	Do not merge
One statutory row vs one non-statutory row	Do not merge
Similar short text with uncertain overlap	Prefer keeping unique evidence
Clearer duplicate observation	Use the better observation as canonical content

A key correction was the stale-row problem: when a clearer observation replaced canonical text and cells, the original row object could still carry the losing photo's elements. The current implementation reassigns the original row whenever a better observation wins, keeping canonical text, cells, geometry, and provenance aligned.

10. Canonical Grounding and Conflict Detection



The fusion stage builds a canonical lookup from every contributing photo-row reference to the canonical text. The structuring engine can consume this lookup directly. If it is called without the precomputed map, its fallback reconstruction prefers row.text and only uses row.elements when row.text is absent.

Statutory conflicts are tracked separately. The current category-aware mechanism covers MRP, unit sale price, net quantity, expiry, manufacture date, batch/lot, FSSAI licence, country of origin, and ingredients. A conflict is recorded when observations belong to the same statutory category, contain materially different numeric payloads, and have sufficiently similar non-numeric declaration text.

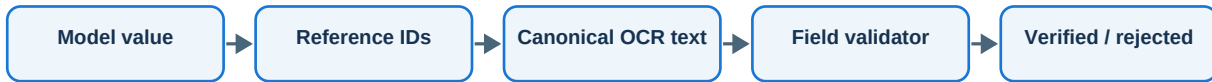
11. GPT-OSS Semantic Structuring



The semantic model receives the evidence organized by photo, row, and cell. Its job is to interpret fragmented or noisy evidence and return a consistent field structure. It is not allowed to become the source of unsupported facts.

- Structured output includes field value, raw observed text, correction state, correction reason, grounding references, and confidence.
- The model may make limited evidence-preserving corrections such as obvious OCR character confusion or joining visibly split fragments.
- Brand name and generic commodity name are kept separate.
- Packaging instructions, addresses, contact information, dates, quantities, nutrition rows, and regulatory labels are protected from being promoted to product identity.
- Generic commodity classification is the only field-level inference explicitly allowed by the structuring prompt.

12. OCR Grounding and Anti-Hallucination Layer



After GPT-OSS returns JSON, each field is checked against the canonical OCR evidence. Unsupported values are not silently accepted. A field can finish as verified, null, not detected, or review depending on the evidence available.

This creates an evidence chain: **OCR observation** → **canonical evidence** → **structured declaration** → **rule input**. That chain is also what makes the result explainable when a reviewer needs to inspect why a value was accepted or rejected.

13. Nutrition and MRP Safety Controls

Nutrition values receive their own grounding check instead of bypassing evidence validation. The present implementation checks the nutrient keyword and the numeric part of the proposed value against nutrition-related OCR evidence.

Known nutrition limitation. The current association check can be too broad: a wrong number may be accepted when it exists elsewhere in the same nutrition block. For example, a carbohydrate number can potentially be accepted as a protein value if both the word “protein” and that number occur in the combined evidence text. This is a known follow-up item for tighter nutrient-to-number association.

The nutrient matcher also has a wording-coverage issue for singular/plural forms such as “Carbohydrate” versus “Carbohydrates”. That produces a safe-side rejection rather than a fabricated value.

MRP tax inclusion is deliberately conservative. The field starts with **inclusiveOfTaxes = null**. Explicit evidence such as inclusive wording sets it to true; explicit exclusive/excluding wording sets it to false; unstated wording remains null.

Evidence	MRP flag
Explicit inclusive wording	true
Explicit exclusive / excluding wording	false
No explicit evidence	null

The regex coverage is not exhaustive for every natural phrase. Wording such as “tax included” or “taxes extra” may currently remain null, which is safe but incomplete.

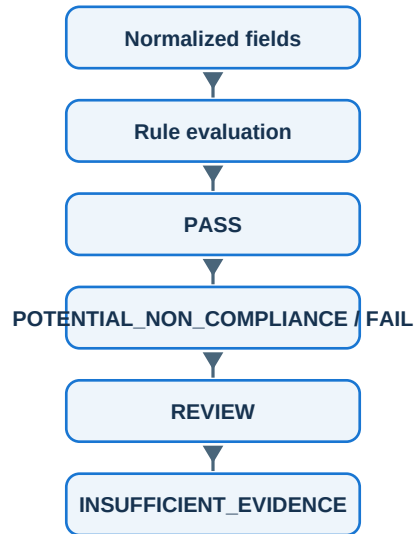
14. Deterministic Fallback



When GPT-OSS is unavailable or fails, the system uses the deterministic fallback extractor. This layer contains explicit pattern matching and spatial pairing for important packaging declarations and marks its output as **deterministic_fallback**.

- MRP, quantity, dates, batch, FSSAI, country, manufacturer/packer/importer and related fields have deterministic handling.
- Spatial label-value pairing reduces some false associations between nearby numeric fields.
- Fallback results still flow into the same rule engine rather than bypassing compliance logic.

15. Legal Metrology Rule Engine



The rule engine is intentionally downstream from extraction and grounding. The LLM does not decide the law; it helps structure observed evidence. The rule layer evaluates the resulting record against application rules and produces structured findings.

Representative checks include

- Responsible party declarations
- Country of origin
- Common / generic commodity name (LM-03)
- Net quantity and unit validity
- MRP and tax-inclusion evidence
- Manufacture / packing and expiry dates
- Consumer-care information
- Unit sale price
- Batch / lot number
- Applicable exemptions

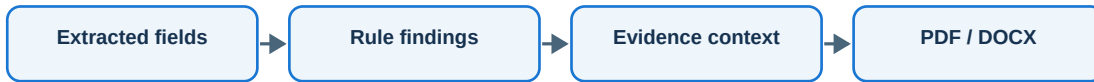
LM-03 safety is handled upstream. The generic commodity field must survive identity filtering and grounding before the rule engine evaluates it. The rule engine itself was kept unchanged during the recent grounding fixes.

16. Cross-Field Consistency Checks

Relationship	Purpose
Net quantity ÷ serving size ≈ servings per container	Detect suspicious serving arithmetic
Manufacture date < expiry / best-before	Detect impossible chronology
Very long shelf-life interval	Escalate to review where appropriate
MRP / unit sale price relationships	Flag economically inconsistent values where normalization permits
Same authoritative field in UI and rule engine	Prevent display/evaluation divergence

The general policy is to downgrade suspicious relationships to review rather than forcing a definitive pass.

17. Reporting and Auditability



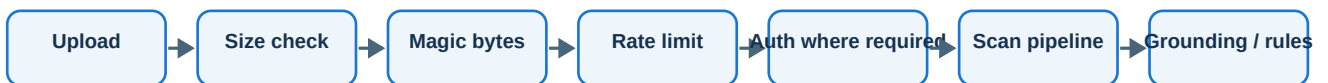
Reporting is part of the compliance workflow. Officer-facing reports are generated in PDF and DOCX formats. The rendered output is intended to keep structured declarations, findings, and explanations connected so a reviewer can understand the basis of the result.

18. Debug and Explainability Architecture



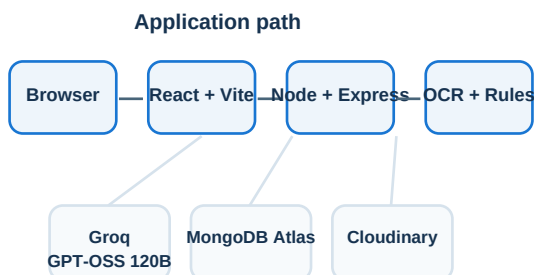
- Stores image metadata and SHA-256 fingerprints.
- Records image dimensions, OCR timing, result counts, and raw OCR detections.
- Retains single-photo and merged extraction records.
- Retains final compliance findings and response data.
- Supports a dedicated debug route for stage-by-stage forensic inspection.

19. Reliability and Security Controls



- Image MIME type and magic-byte validation.
- Default 10 MB per-image upload limit in the validator.
- In-memory rate limiting for scans and authentication endpoints.
- Explicit OCR failure responses.
- Environment-based storage of the Groq API key.
- Production error middleware that hides internal stack details.
- Cloudinary with local-storage fallback for evidence images.

20. Data and Storage Architecture



Rules are stored as structured documents containing rule code, field, category, description, validation type and parameters, effective dates, source reference, and severity. Operational records store user metadata, scan timestamp, extracted fields, findings, overall status, mismatch information, and evidence-image references.

21. Prototype Development Lifecycle

Phase	Focus	Outcome
Phase 1	OCR proof of concept	Established the initial React/Vite → Node/Express → FastAPI/PaddleOCR path.
Phase 2	OCR stabilization	Separated the OCR service, added service health handling, model reuse, and cleaner runtime behavior.
Phase 3	Geometry and reconstruction	Introduced preprocessing, confidence-aware OCR retention, spatial ordering, line/row reconstruction.
Phase 4	Deterministic extraction	Expanded structured extraction for packaging declarations and added validators, spatial pairing, and more.
Phase 5	Performance and multi-photo OCR	Introduced concurrent OCR across photos, three-tier image routing, full-resolution tiling, and longer runs.
Phase 6	Identity and compliance safety	Addressed identity collisions, instructional text, generic-name fallback risk, field-level validation, and more.
Phase 7	GPT-OSS semantic mapping	Moved semantic ambiguity handling to GPT-OSS 120B while keeping evidence and statutory logic intact.
Phase 8	Grounding and conflict safety	Fixed stale canonical-row evidence, added rowLookupMap grounding, statutory conflict detection, and more.

22. OCR Model Evaluation and Performance Work

RapidOCR / ONNX-style alternatives were explored during development as lighter-weight options. Their initialization was attractive, but recognition quality on the project's packaging cases was weaker, so PaddleOCR remained the primary OCR engine.

An early benchmark recorded roughly 29.57 seconds of server latency for a 960×1280 image. Other runs reported roughly 40–50 seconds in heavier configurations. The investigation showed that resolution, tiling, CPU inference, request sequencing, and infrastructure all contributed to latency; file size alone was not a sufficient explanation.

The current performance path is measured as separate stages: upload → preprocess → OCR → reconstruction → fusion → GPT-OSS → rules → report. That makes it possible to improve the actual bottleneck instead of changing components without evidence.

23. Verification and Regression Testing

Suite	Result	Coverage
test_grounding_conflict_regression.js	20 / 20	Stale row evidence, canonical lookup, statutory conflicts, nutrition grounding, MRP tax handling
test_multi_photo_fusion_suite.js	31 / 31	Deduplication, stable row IDs, grounding, identity separation and fusion safety
test_structuring_engine.js	6 / 6	Invention prevention, row concatenation, packaging rejection and fallback behavior
test_golden_regression.js	16 / 16	Broader smoke/regression coverage
Offline total	73 / 73	Independently verified offline suites

The live Groq test depends on a working API key and network path. Its execution therefore remains environment-dependent and should be run in the same environment used for the live demonstration.

24. Real-Product Testing Experience

Earlier controlled photographs exposed a mixed result: some statutory fields such as MRP, batch and FSSAI were recovered, while identity, quantity, servings, dates, and parts of manufacturer/packer information were less reliable. The final state correctly avoided turning the uncertain generic-name path into an automatic LM-03 pass.

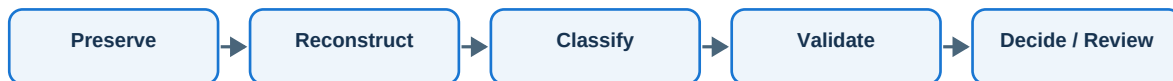
Those tests also showed why end-to-end tracing matters. A real-image failure can originate in OCR, reconstruction, candidate generation, fusion, model mapping, or grounding. The correct diagnostic sequence is:

PaddleOCR raw output → tokens/bounding boxes → row reconstruction → normalized rows/cells → candidate generation → multi-photo fusion → GPT-OSS input → GPT-OSS output → grounding → normalized fields → rule engine → LM finding → report.

25. Present Limitations and Follow-Up Work

- Nutrition grounding still needs tighter nutrient-to-number association so that a value from one row cannot be accepted for a different nutrient merely because both appear in the same evidence block.
- Nutrition wording coverage should recognize common singular/plural variants such as “Carbohydrate” and “Carbohydrates”.
- MRP tax wording coverage can be expanded for phrases such as “tax included” and “taxes extra”, while keeping the safe null default.
- Statutory conflict fixtures should be expanded so every supported category has a dedicated regression case.
- Legacy Gemini compatibility code and documentation can be removed or consolidated after the GPT-OSS path is fully settled.
- The README still reflects the earlier OCR proof-of-concept and can be refreshed to describe the mature pipeline.
- Ultra-large tiled OCR remains a potentially expensive CPU path and should be optimized only after stage timing is measured on the real demo hardware.

26. Engineering Principles



The project deliberately avoids two failure modes: **guessing missing data** and **rejecting useful evidence too early**. When evidence is weak, the correct outcome is an explicit unresolved state rather than a fabricated value.

The same design principle applies to the whole system: keep evidence rich enough for later reasoning, but keep compliance logic deterministic and inspectable.

27. Project Positioning

DrishtiScan is best understood as an evidence-preserving compliance workflow rather than a simple OCR-plus-AI application. Its main technical contribution is the separation of evidence capture, spatial reconstruction, multi-photo reconciliation, semantic interpretation, deterministic grounding, and rule evaluation.

That separation allows the system to improve individual stages independently while keeping the final compliance path explainable. The prototype is now at the stage where the main engineering work is no longer building the pipeline from scratch, but validating it against real packaging conditions, tracing each remaining failure to its exact stage, and applying targeted improvements without sacrificing safety.

Photo → OCR → spatial reconstruction → multi-photo fusion → semantic structuring → grounding → rule engine → explainable result.